

RAG - Vectors and Embeddings

Naresh Chaurasia

Version 1.0, 12/Jun/2026: First Edition

Table of Contents

1. The Problem	1
2. The Concept	2
3. How It Works	3
3.1. From Raw Text to a Vector	3
3.2. Architecture of the Retrieval Flow	3
3.3. How Similarity Is Measured	4
4. Key Components	5
4.1. The Embedding Model	5
4.2. The Vector Database	7
4.3. The Chunking Step	7
4.4. The RAG Pipeline	8
5. How It Solves the Problem	9
6. Summary	10
7. Quick Self-Check Questions	11
7.1. Foundational Questions	11
7.2. Intermediate Questions	11

Chapter 1. The Problem

Imagine you are building a customer support system for an e-commerce company called ShopEasy. ShopEasy receives around 50,000 customer queries every day. Customers ask things like:

- "Where is my order?"
- "How do I return a damaged product?"
- "I haven't received my refund yet."
- "My package is missing."

ShopEasy already has a knowledge base of 10,000 previously answered support tickets. The engineering team wants to build a system that automatically finds the most relevant past answer for each new incoming query, so that the support agent — or an automated chatbot — can respond instantly.

The team first tries a traditional keyword-based search. They index all 10,000 tickets using SQL and query them with exact word matches.

The result? A query like "Where is my order?" matches tickets containing those exact words. But "My package is missing" returns nothing useful — even though it describes the same problem. The word "order" does not appear in that sentence, so SQL finds no match.

The team counts the failures. Out of 1,000 test queries, 340 return either no result or a completely irrelevant one. That is a 34 percent failure rate, which makes the chatbot practically useless for a large portion of customers.

The core problem is this: traditional search systems compare words literally. They do not understand meaning. "Order", "package", "shipment", and "delivery" all refer to the same concept in a customer support context, but a keyword-based system treats them as entirely different strings.

To solve this, ShopEasy needs a way to **represent the meaning of text numerically — so that sentences with similar meaning produce similar numbers**, and those similar numbers can be compared quickly at scale. This is precisely where vectors and embeddings come in.

Chapter 2. The Concept

Before introducing any technical terms, consider a simpler analogy.

Suppose you want to organise all the items in a supermarket not by their names alphabetically, but by how similar they are in terms of use, taste, and nutrition. You place milk and yoghurt close together. You place apples and pears near each other. You place a motor oil can far away from the food section.

Now imagine that every product is given a coordinate in this space — a set of numbers that together describe its position. Milk might be at coordinates (8, 2, 9) on axes representing, say, dairy-content, sweetness, and protein. Yoghurt might be at (7, 3, 8) — very close. Motor oil would be at (0, 0, 0) — far away in every direction.

If someone asks "What is similar to milk?", you find the coordinates of milk and look for whatever else has nearly the same coordinates. You do not search by the word "milk". You search by position in this meaning-space.

This is exactly what vectors and embeddings do for text, images, audio, and other data.

A **vector** is a list of numbers that represents an item — a word, a sentence, an image, or a product — in a multi-dimensional mathematical space. An **embedding** is a specific kind of vector produced by a machine learning model that has been trained on large amounts of data. Because the model has learned from context and usage patterns, the vectors it produces group similar things close together in that space.

When ShopEasy converts each support ticket into an embedding, tickets about missing packages, lost orders, and shipping delays all land near each other in this multi-dimensional space — regardless of the exact words used. Matching a new query now becomes a matter of finding the nearest point in space, not the nearest matching word.

Chapter 3. How It Works

3.1. From Raw Text to a Vector

The process of turning a customer query into a vector follows a consistent sequence of steps. Using the ShopEasy example:

Step 1: Input text is passed to an embedding model. The model is a neural network pre-trained on massive text corpora. Popular choices include models based on BERT, GPT, or specialised sentence-transformers. ShopEasy sends the text "My package is missing" to such a model.

Step 2: The model processes the text and outputs a vector. The output is not a single number. It is a list of numbers—typically between 300 and 1,536 dimensions depending on the model. Each position in the list captures some abstract feature of the text. The complete list together encodes the overall meaning.

Step 3: The vector is stored. ShopEasy stores this vector in a vector database alongside the original ticket text and its metadata (ticket ID, resolution, category).

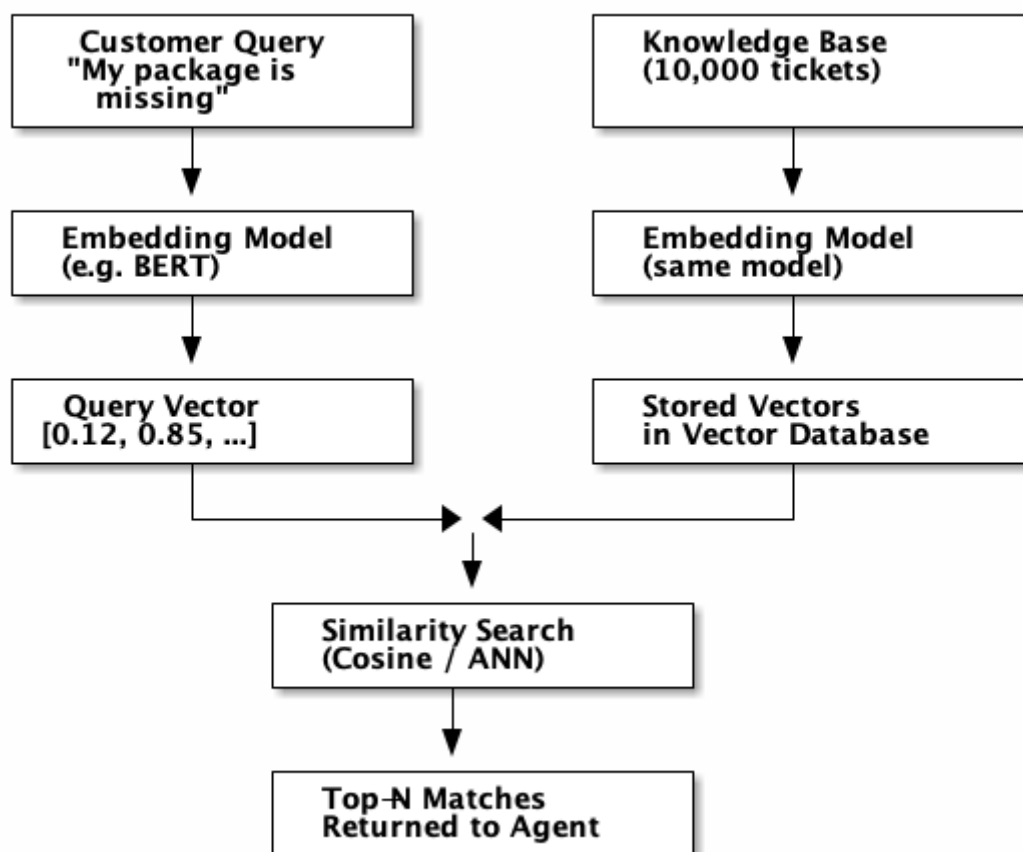
Step 4: At query time, the incoming question is also converted to a vector. The same embedding model is used. "Where is my order?" becomes a new vector.

Step 5: Similarity is computed. The new vector is compared with every stored vector. Items whose vectors point in similar directions—meaning their content is semantically related—receive high similarity scores.

Step 6: The top matches are returned. The system retrieves the top-N most similar tickets and presents them to the support agent or chatbot.

3.2. Architecture of the Retrieval Flow

The following diagram shows the full end-to-end flow for ShopEasy's support system.



3.3. How Similarity Is Measured

Once both the query and the stored tickets have been converted to vectors, a mathematical formula is used to decide how close two vectors are. The most common method is **cosine similarity**.

Cosine similarity measures the angle between two vectors in space. If two vectors point in nearly the same direction, the angle between them is small, and the cosine value is close to 1. If they point in completely different directions, the cosine value is close to 0.

In the ShopEasy case, the vectors for "My package is missing" and "I haven't received my shipment" would produce a cosine similarity close to 0.91, for example. The vectors for "My package is missing" and "I want to cancel my subscription" would produce a cosine similarity closer to 0.20. The system uses these scores to rank and return the most relevant results.

Two other metrics are also used in practice:

Euclidean distance measures the straight-line distance between two points. Smaller distance means more similar.

Dot product measures both the direction and the magnitude of two vectors. It is faster to compute but sensitive to how long the vectors are.

Chapter 4. Key Components

4.1. The Embedding Model

▼ *Embedding - boAt Airdopes 141 earbuds*

Term: 'boAt Airdopes 141 earbuds'

Embedding vector (384 dims):

```
[-3.69760394e-02 1.85434446e-02 3.53143131e-03 -1.34988185e-02 -3.70238759e-02 -7.74916261e-02 5.95261827e-02 2.35074032e-02 -1.26353428e-01 2.14693043e-03 4.72359173e-02 -7.82370046e-02 -1.13589503e-02 8.67439061e-03 -4.45370264e-02 8.70122612e-02 1.16957435e-02 -4.77400534e-02 6.97562820e-04 -5.03108930e-03 -5.09332828e-02 1.23601437e-01 3.09766885e-02 -2.13817004e-02 -7.34671727e-02 3.64338569e-02 -2.47838013e-02 -1.84325688e-02 -5.31779565e-02 -6.22104555e-02 -8.20727553e-03 1.61549225e-02 2.94988062e-02 -3.73995155e-02 3.65963280e-02 -1.88892931e-02 7.47754145e-03 -4.77676950e-02 -1.87549796e-02 5.76434918e-02 -6.81604892e-02 3.01790461e-02 -2.17413176e-02 6.30961126e-03 -3.45630385e-02 -1.46961194e-02 -9.44780260e-02 -1.27401287e-02 9.14171040e-02 1.18977278e-01 6.82477504e-02 -3.99155915e-02 2.93350182e-02 3.47527303e-02 5.33743808e-03 -3.91390994e-02 -1.29945517e-01 4.48769331e-02 -3.17482231e-03 3.19618136e-02 9.38101411e-02 4.82252464e-02 6.12887926e-03 -6.32545277e-02 -2.80179083e-02 -1.42266471e-02 -4.73944172e-02 1.27016418e-02 2.68113893e-02 -3.72365117e-03 -4.13880662e-05 5.13208956e-02 -3.72546390e-02 1.55998124e-02 2.88956389e-02 7.80661255e-02 2.20242608e-02 -8.08212087e-02 6.19543977e-02 3.29761878e-02 -1.04920015e-01 -1.32882074e-01 -3.02411988e-02 -6.43793568e-02 3.15972641e-02 -7.43248640e-03 6.79200515e-02 8.26381799e-03 -9.09662619e-02 7.44155496e-02 -2.38816291e-02 -1.82421654e-02 -2.21120622e-02 -4.21560146e-02 3.85858305e-02 1.74314827e-02 -4.93220612e-02 5.29378578e-02 -4.88174483e-02 4.98232879e-02 5.73906228e-02 -4.75357380e-03 7.69470632e-02 -6.70122728e-02 -1.64380278e-02 -2.21956801e-02 -4.53670993e-02 -1.16159646e-02 7.88903516e-03 -4.74597923e-02 -6.17887825e-02 -1.48162423e-02 5.03278449e-02 -2.51443069e-02 -7.33476505e-03 -2.75710924e-03 -1.18556187e-01 -8.20320286e-03 -2.67467126e-02 -3.41231264e-02 8.98983702e-02 -1.86650287e-02 1.15311854e-01 -1.17746321e-02 -5.21053821e-02 -3.55394445e-02 -4.94421460e-02 1.72789698e-33 -2.20818669e-02 1.15931844e-02 -4.64938991e-02 -1.86116900e-02 4.68698936e-03 -4.04291041e-02 -8.63722041e-02 3.01947258e-02 2.92726699e-02 8.51510093e-02 -1.37043446e-01 6.45986274e-02 -2.37453021e-02 -4.31761518e-02 8.09689462e-02 -9.59970802e-02 -4.59100083e-02 7.71892350e-03 -8.64770934e-02 -1.10400710e-02 -3.31166051e-02 7.30299344e-03 -3.44322622e-02 6.62986711e-02 9.11024809e-02 4.39517051e-02 -2.81785373e-02 -5.91483004e-02 -1.93895735e-02 7.89946467e-02 2.07399111e-03 -4.08362113e-02 1.87863484e-02 -2.63531227e-03 -2.37643030e-02 2.54794862e-02 5.64533137e-02 -7.37445131e-02 -8.36907700e-02 -6.81670755e-02 9.13058072e-02 2.35925964e-03 1.23386784e-02 1.10926077e-01 1.91296842e-02 6.20493926e-02 -2.84207035e-02 7.34557286e-02 7.51585588e-02 -3.53894979e-02 -5.79257719e-02 4.51604538e-02 -5.29224277e-02 6.58950815e-03 5.46404608e-02 2.54577436e-02 2.90146321e-02 1.55404545e-02 1.85622890e-02 -2.97424514e-02 2.41893157e-02 1.24229230e-01 7.15959296e-02 -4.65065911e-02 -2.79330444e-02 5.45501411e-02 4.98271063e-02 -6.16391003e-02 7.71630323e-03 -8.64698291e-02 -2.19651964e-02 2.07634009e-02 2.49816999e-02 -5.61580248e-02 -3.47288325e-03 8.66219494e-03 1.61407515e-02 -3.45494086e-03 -8.05987220e-04 -3.08073424e-02 -6.08902611e-02 6.92139268e-02 -4.61943261e-03 3.74542549e-02 7.69535080e-02 -5.52491210e-02 1.68394460e-03 -1.03761069e-02 -2.24877335e-02 2.21531000e-03 -1.26960594e-03 2.64736842e-02 -4.32746708e-02 -1.03916012e-01 1.36260381e-02 -2.00572783e-33 2.35559624e-02 2.06098314e-02 -1.46497441e
```

-02 3.47382203e-02 2.49993391e-02 8.65106285e-02 8.76758844e-02 6.03244640e-02 -4.46504317e-02 -4.83844765e-02 -3.39821316e-02 -2.43248441e-03 4.95438697e-03 4.10716571e-02 1.03718616e-01 -1.03256619e-02 1.20343631e-02 -2.17544865e-02 3.52312699e-02 -2.62917113e-02 1.39866313e-02 3.73446825e-03 6.75093308e-02 7.80695900e-02 -6.39061630e-02 5.56943156e-02 1.51634996e-03 -9.14181098e-02 4.91682673e-03 -9.92201082e-03 -1.77205596e-02 5.19361608e-02 2.80626770e-02 3.95041257e-02 -6.75119907e-02 -3.59348767e-02 8.72057602e-02 1.14874430e-02 -6.21336661e-02 -4.52502184e-02 5.16595505e-02 -5.12490869e-02 -7.50252046e-03 -4.88258339e-02 -1.12235695e-01 -3.63791175e-02 -1.08271127e-03 -2.17063986e-02 -3.58758010e-02 -5.13227843e-02 7.52834231e-02 2.86955736e-03 2.21113320e-02 -1.95685569e-02 -1.66609678e-02 1.71948932e-02 -2.62858737e-02 -7.54329488e-02 2.91415844e-02 3.21521983e-02 8.39431062e-02 -2.40926277e-02 -5.33761084e-02 5.62678799e-02 -1.96475592e-02 7.36931860e-02 1.42352637e-02 1.69935171e-02 -5.72337508e-02 -2.66671944e-02 1.10709788e-02 -4.79374357e-05 4.11238372e-02 5.67541048e-02 1.17515046e-02 -3.12593058e-02 -4.38800603e-02 7.07683992e-03 9.95252356e-02 3.21474150e-02 -6.16004989e-02 -1.18355849e-03 9.49603226e-03 7.30335712e-02 3.63010429e-02 -2.97076423e-02 6.39803940e-03 -1.88931320e-02 -3.68248299e-02 2.75792107e-02 2.93323118e-02 5.76998219e-02 -2.56654508e-02 6.41029626e-02 9.50524025e-03 -1.65781309e-08 2.90664639e-02 1.17969528e-01 4.12066095e-02 1.24668693e-02 1.32800955e-02 -1.96235199e-02 -1.18202707e-02 9.31769386e-02 -5.17469235e-02 -4.91916761e-02 8.30318332e-02 4.18654736e-03 6.16851309e-03 4.30718809e-02 6.01215623e-02 -4.15838100e-02 -2.54204609e-02 5.93781881e-02 -4.33510616e-02 -1.38071150e-01 -4.45307326e-03 9.13172960e-02 5.99381467e-03 -1.00276582e-02 -7.61796385e-02 1.18484162e-02 5.91930784e-02 5.45757674e-02 5.96653037e-02 1.70418806e-02 -3.28704938e-02 2.89600994e-02 -1.28646597e-01 -7.77994171e-02 -8.42624530e-03 -3.69204544e-02 -2.74380930e-02 5.62337600e-03 -6.15793243e-02 5.62058277e-02 -7.13686422e-02 -5.59411244e-03 -1.95166785e-02 1.89625807e-02 4.46509942e-02 6.83624239e-04 8.29214752e-02 -4.64062765e-03 -6.87463433e-02 -4.00133757e-03 -3.46610025e-02 5.98977618e-02 4.55175824e-02 2.63904352e-02 -1.94419008e-02 7.01170564e-02 -1.65435355e-02 -8.75598053e-04 -1.04880564e-01 1.03548355e-02 -1.07386060e-01 -2.64987871e-02 5.27760247e-03 1.18314572e-01]

The embedding model is the engine that converts raw data into vectors. It is a trained neural network that has learned, from enormous amounts of text (or images, or audio), which things tend to appear in similar contexts.

For text, models based on the **Transformer architecture**—such as BERT and its derivatives, or GPT-based models—are the most common today. They produce **contextual embeddings**, meaning the vector for the word "bank" will be different when it appears in the sentence "I deposited money at the bank" versus "I sat on the river bank". The surrounding context shapes the output vector.

Older models such as **Word2Vec** and **GloVe** produce a single fixed vector for each word regardless of context. They are lighter-weight and faster, but less accurate on nuanced language tasks.

For ShopEasy, using a sentence-level embedding model—one that reads the entire support ticket and produces a single vector for the whole sentence—is more appropriate than word-level models. Sentence transformers such as those from the [sentence-transformers](#) library are designed exactly for this.

4.2. The Vector Database

A vector database is a storage system optimised specifically for storing and querying vectors. Unlike a traditional relational database that is designed to answer questions like "give me all records where city equals Mumbai", a vector database is designed to answer questions like "give me the 10 records whose vectors are closest to this new vector."

Standard relational databases store structured rows and columns. They are fast at exact lookups and range queries on numbers and strings. But they have no built-in concept of vector distance or similarity search. Running similarity queries on them requires computing distances row by row, which is extremely slow at scale.

Vector databases solve this by building specialised index structures — such as **HNSW** or **IVF** — at write time, so that queries at read time can be answered in milliseconds even across millions of records.

HNSW builds a layered graph connecting similar vectors, so a search "hops" through the graph toward the nearest matches instead of checking everything — fast, but memory-heavy.

IVF groups vectors into clusters ahead of time; a search only looks inside the few clusters closest to the query — lighter on memory, slightly less precise. Both trade a bit of accuracy for massive speed gains at scale, which is why real vector databases use them instead of brute-force comparison.

Common vector databases in production use today:

FAISS (Facebook AI Similarity Search) is an open-source library, not a full database. It runs **in memory** on a single machine. It is the right choice for **prototyping** or for systems where the data fits comfortably in RAM. ShopEasy uses FAISS during development.

Pinecone is a managed **cloud service**. It handles scaling, replication, and indexing automatically. ShopEasy migrates to Pinecone when moving to production, because it eliminates the operational overhead of managing the vector index.

Weaviate is a vector database that also supports traditional keyword search, allowing **hybrid queries** — for example, "find tickets that mention the word refund AND are semantically similar to this query."

4.3. The Chunking Step

Documents are rarely small enough to convert into a single vector effectively. A 10-page product manual, for example, cannot be meaningfully captured as one vector — the vector would be an average of too many different topics and would not be informative for similarity search.

The solution is **chunking**: splitting a document into smaller pieces before embedding. Each chunk — perhaps a paragraph or a sliding window of 200 words — is converted into its own vector.

The system then stores thousands of chunk-level vectors, each pointing to its source document and its position within it.

For ShopEasy's support tickets, each ticket is short enough to be treated as a single chunk. But if they later index the full product documentation or return policy PDFs, those documents would need to be chunked first.

4.4. The RAG Pipeline

The full workflow that ShopEasy builds is an instance of a pattern called **Retrieval-Augmented Generation (RAG)**. In this pattern:

1. A query arrives.
2. The query is converted to an embedding.
3. Similar documents are retrieved from the vector database.
4. The retrieved documents are passed as context to a large language model (LLM).
5. The LLM generates a response that is grounded in the retrieved content rather than relying purely on what it learned during training.

The vector retrieval step is the foundation. Without accurate retrieval, the LLM receives irrelevant context and produces a poor response. Embeddings and the vector database are what make retrieval fast, accurate, and scalable.

Chapter 5. How It Solves the Problem

Recall the original problem. ShopEasy's keyword-based SQL search failed on 34 percent of test queries — 340 out of 1,000 — because it could not match queries about the same concept when different words were used.

With the embedding-based vector search system in place, ShopEasy re-runs the same 1,000 test queries. The failure rate drops to under 4 percent — 38 out of 1,000. Queries like "My package is missing", "I haven't received my order", and "Where is my shipment?" all retrieve the same set of relevant tickets, because their embeddings are geometrically close in vector space.

The table below summarises the before-and-after contrast across key dimensions.

Dimension	Before: SQL Keyword Search	After: Embedding-Based Vector Search
Match strategy	Exact word match — "order" must appear in the ticket	Semantic match — "order", "package", "shipment" treated as equivalent
Query failure rate	34 percent (340 / 1,000 test queries)	Under 4 percent (38 / 1,000 test queries)
Handles synonyms	No	Yes
Handles phrasing variation	No	Yes, because meaning is encoded, not words
Speed at 500,000 tickets	Degrades with full-table scan	Millisecond response with ANN indexing
Handles multilingual queries	Only if exact translation is stored	Partially, if a multilingual embedding model is used
Requires exact query formulation	Yes	No — approximate meaning is sufficient
Data types supported	Text with structured schema	Text, images, audio, video — any data the embedding model supports

The improvement is not just in failure rate. The system is now extensible. ShopEasy can embed product images, so a customer who uploads a photo of a damaged package gets matched to relevant return-procedure tickets. They can embed audio from phone calls, once transcribed, and route those calls to the right department automatically. The foundation — convert data to vectors, store them, search by similarity — generalises across every data type.

Chapter 6. Summary

Vectors and embeddings solve one of the most fundamental limitations in information retrieval: the inability of traditional systems to understand meaning rather than just match words. An embedding is a list of numbers produced by a machine learning model that captures the semantic content of data — text, images, audio, or video. Items with similar meaning end up with similar vectors, positioned close together in a high-dimensional mathematical space. This makes it possible to search by meaning rather than by exact keyword.

For engineering students building intelligent applications, the practical implication is straightforward. Whenever your system needs to retrieve relevant information in response to a user query, and whenever the exact wording of the query cannot be predicted in advance, an embedding-based retrieval layer is the appropriate tool. Combining that retrieval layer with a generative model — the RAG architecture — produces systems that are both grounded and flexible. The vector database is the infrastructure that makes this retrieval fast enough to work in real time at production scale. Mastering how embeddings are generated, stored, indexed, and queried is therefore a foundational skill for building modern AI-powered applications.

Chapter 7. Quick Self-Check Questions

7.1. Foundational Questions

1. **What is a vector, and why is it useful for representing text?**

Hint: Think about what it means to represent something as a list of numbers, and how numbers can be compared using distance.

2. **What is the difference between an embedding and a raw vector?**

Hint: Consider where the numbers come from—are they arbitrary, or are they produced by a system that has learned something?

3. **Why does cosine similarity use the angle between vectors rather than the length of the vectors?**

Hint: Think about a case where two documents have the same words but one is much longer. Should they have the same similarity score?

4. **What is the role of a vector database, and how does it differ from a relational database?**

Hint: Consider what kind of query each type of database is optimised to answer quickly.

5. **In the ShopEasy example, why does "My package is missing" match the same tickets as "Where is my order?" after embedding, even though they share no words?**

Hint: Think about what the embedding model was trained on and what kind of information it captured about how these phrases are used.

7.2. Intermediate Questions

1. **Why is chunking necessary before embedding long documents, and what trade-offs does chunk size introduce?**

Hint: Consider what happens to the vector if a single chunk contains too many different topics. Then consider what happens if chunks are too small.

2. **Compare HNSW and IVF as indexing strategies for a vector database. In what scenario would you choose one over the other?**

Hint: Think about how much memory is available, how often data is updated, and whether accuracy or speed is the higher priority for the application.

3. **A team decides to use Word2Vec embeddings instead of a sentence-transformer for their customer support retrieval system. What specific failure modes might they encounter?**

Hint: Consider how Word2Vec handles context—does the same word get a different vector depending on its surrounding words?

4. **In a RAG pipeline, why is the quality of the retrieval step more important than the power of the generative model?**

Hint: If the retrieved chunks contain no relevant information, what can the language model realistically do with them?

5. **Weaviate supports hybrid search—combining keyword search with vector search. Describe a real-world scenario where hybrid search would outperform pure vector search alone.**

Hint: Think about a case where the user is searching for a specific product code, serial number, or proper noun that has no semantic equivalent—something that must be matched exactly.